

Quantized storage and loaded model memory

The retained Olympus fixture stores smaller integer-weight artifacts but loads the same 138,112 bytes of float32 parameters for all three variants. A new separate-process measurement records model loading, resident memory and forward timing without mixing the variants' cumulative process peaks.

This is an executed engineering measurement on the existing 36-record constructed fixture. It is not a completed MLSys or ACL research study. No new model was trained, no protected experiment was dispatched, and no model was promoted.

Protocol and actual measurements

The [protocol](#) was written before the nine measurement processes. Each variant runs in a fresh Python interpreter three times. The order rotates across repetitions. Both torch thread counts are one. Loading is timed after imports; a fixed 64-token CPU forward workload receives two warmup calls and seven timed calls per process. The exact token count and input digest are retained in every trial.

Measurement	Float32 artifact	Stored int4 artifact	Stored int8 artifact
Artifact bytes	144,790	32,277	49,493
Loaded parameter bytes	138,112	138,112	138,112
Loaded buffer bytes	0	0	0
Median load time, ms	5.450	12.868	6.536
Median resident increase after load, MiB	3.918	7.258	5.820
Median resident memory after forward, MiB	269.410	270.105	269.723
Median of per-process median forward time, ms	0.336	0.263	0.348

All values come from the [nine native trials and full results](#). Storage reductions relative to the serialized float state are 77.708% for int4 and 65.817% for int8. The parameter-storage identity is exact for these artifacts. The loader reconstructs float32 tensors; no integer inference kernel is present. The apparent timing differences in this small shared CPU run are descriptive samples and do not establish a speedup or task-quality advantage.

The baseline and quantized paths both load state into the same TinyCausalLM configuration. Optimizer state is excluded from the float32 comparison. Each input artifact must match the SHA-256 in the earlier retained probe before it is loaded. `torch.load(..., weights_only=True)` is used. All original checkpoint, manifest, int4 and int8 artifacts remain byte-for-byte unchanged.

Interpretation and limits

Linux `/proc/self/statm` supplies approximate resident-page counters, which include shared runtime libraries. These are not proportional set size, GPU memory, energy or monetary cost. `getrusage` process high-water counters are reported separately; values sampled by these interfaces are not an exact instantaneous reconciliation. The imported runtime dominates a model this small. OS file caches are not cleared, so load times are not cold-storage latencies. The [kernel's process-memory documentation](#) explains the asynchronous RSS accounting and the more precise, slower `smaps` snapshot. This probe keeps those limitations visible rather than presenting its resident-page counters as exact allocation accounting.

Model-parameter bytes are the clearest conclusion: reducing the saved artifact does not reduce the parameters' loaded float32 representation in this loader. Representative tasks, larger models, meaningful quality baselines, isolated hardware runs and a justified systems contribution remain necessary for the conference research routes. The [MLSys2027 call](#) also requires actual author contributions and disallows LLM-only generated paper content except its stated illustrative/experimental uses. This technical evidence note is preparation for author-led research, not a submission-ready paper.

Reproduce

From the repository root, in the supported Python 3.14 environment:

```
PYTHONPATH=. python scripts/isolated_quantization_probe.py \  
docs/evidence/quantization-timing-2026-10-07 /tmp/olympus-isolated-new
```

The output directory must be new. The protocol, all nine raw trial records, summary, source hashes, artifact hashes, model dtypes, timing samples and platform are saved. The retained run used Python 3.14.7 and torch 2.11.0+cpu on Linux. Ruff 0.16.10 passed for the new script. An earlier local run reached the summary step and exposed a path-formatting error; that error was corrected and the entire nine-process run was executed again. Only the successful corrected run is cited.